

Design and Implementation of Reliable Data Transfer Protocols in C

Gowtham Prasad, 12226856

Abstract—This report provides a comprehensive analysis of two reliable data transfer protocols—the Alternating Bit Protocol (ABP) and Go-Back-N (GBN)—implemented in C for a simulated network environment. Both protocols ensure error-free, in-order data delivery between sender (A) and receiver (B) under packet loss, corruption, and delays. The report details protocol design, implementation challenges, performance under varied network conditions, and comparative efficiency.

I. INTRODUCTION

Reliable data transfer (RDT) is critical for applications requiring guaranteed delivery over unreliable channels. This work implements ABP (stop-and-wait) and GBN (sliding window) protocols, leveraging sequence numbers, checksums, and timers to handle network impairments. The network emulator introduces configurable loss (0–20%), corruption (0–20%), and delays, enabling rigorous validation.

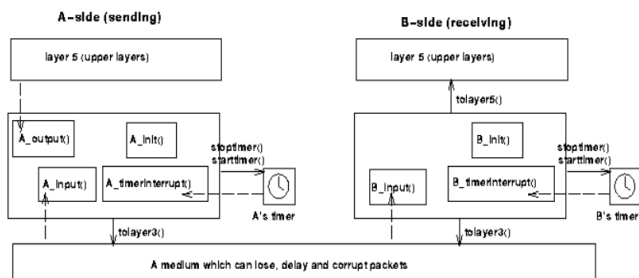


Fig. 1. Overall structure of the environment.

II. PROTOCOL DESIGN AND IMPLEMENTATION

A. Alternating Bit Protocol (ABP)

ABP uses a 1-bit sequence number (0/1) to track packet acknowledgment states.

Sender Workflow:

- **New Packet Transmission:** `A_output()` sends a packet if no unacknowledged data exists.
- **Retransmission Handling:** `A_timerinterrupt()` resends the last packet after a timeout (e.g., 15 seconds).

- **Acknowledgment Processing:** Valid ACKs advance the sequence number; corrupted or NAKed packets trigger retransmission.

Receiver Workflow:

- **Validation:** `B_input()` checks packet checksums and sequence numbers.
- **ACK/NAK Generation:** Valid in-order packets trigger ACKs; corrupted or out-of-sequence packets send NAKs.

```
1 void A_timerinterrupt() {
2     if (SenderA.state != WAIT_ACK) {
3         printf("A_timerinterrupt: not waiting ACK...ignore event.\n");
4         return;
5     }
6
7     printf("A_timerinterrupt: resend last packet: %s.\n",
8           SenderA.last_packet.payload);
9
10    tolayer3(0, SenderA.last_packet);
11    starttimer(0, SenderA.estimated_rtt);
12 }
```

Fig. 2. Code snippet of `A_timerinterrupt()`.

B. Go-Back-N (GBN)

GBN uses a sliding window of size 8 to pipeline packets, improving throughput over ABP.

Sender Workflow:

- **Cumulative ACKs:** Advances the window upon receiving ACKs for the oldest unacknowledged packet.
- **Bulk Retransmission:** Timeouts resend all packets to the current window.

Receiver Workflow:

- **In-Order Delivery:** Discards out-of-sequence packets, sending ACKs for the last correctly received packet.

III. RESULTS

A. Alternating Bit Protocol (ABP)

1. In Fig 3, this simulation tests the Stop-and-Wait protocol using `abt_prog2.c` with 20 messages, 20% packet loss, 20% corruption, and an average

message interval of 10 time units. It evaluates how the protocol performs under unreliable network conditions.

```
168 EVENT time: 270.968170, type: 2, fromlayer3 entity: 0
169 A_input: acked.
170 Simulator terminated at time 270.968170
171 after sending 20 msgs from layer5
```

Fig. 3. ABP Test 1 Results.

2. In Fig 4, this simulation of the Stop-and-Wait protocol (abt_prog2.c) was run with 20 messages, no packet loss or corruption, and a message interval of 10 time units. It evaluates the protocol under ideal network conditions. The protocol performed reliably, with each sent packet being acknowledged and successfully delivered. However, due to Stop-and-Wait's sequential nature, any delay in acknowledgment caused the sender to drop new messages that arrived before the previous one was acked. Even without network errors, this resulted in several dropped messages and underutilized channel capacity.

```
148 EVENT time: 207.934280, type: 2, fromlayer3 entity: 0
149 A_input: acked.
150 Simulator terminated at time 207.934280
151 after sending 20 msgs from layer5
```

Fig. 4. ABP Test 2 Results

3. In Fig 5, this simulation of the Stop-and-Wait protocol (abt_prog2.c) was run with 20 messages, no packet loss, 20% corruption, and a message interval of 10 time units. It examines how the protocol handles corrupted packets in an otherwise reliable network. The simulation shows that corruption alone can significantly delay successful delivery. Several packets required multiple retransmissions before being acknowledged, such as ffffffff, which experienced repeated corruption. Because the sender waits for an ACK before proceeding, many later messages were dropped. Although the receiver sends NAKs for corrupted packets, the sender ignores them and only retransmits after timeouts, slowing recovery..

```
1 EVENT time: 210.360443, type: 2, fromlayer3 entity: 0
2 A_input: acked.
3 Simulator terminated at time 210.360443
4 after sending 20 msgs from layer5
```

Fig. 5. ABP Test 3 Results

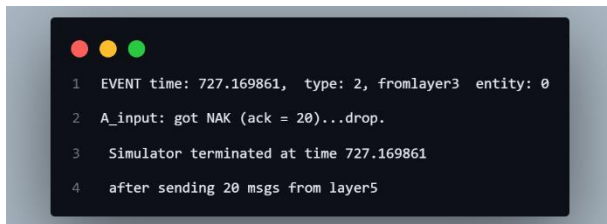
B. Go-Back-N (GBN)

1. In Fig 6, this simulation tests the Go-Back-N (GBN) protocol with 20 messages, 20% packet loss, 20% corruption, and an average message interval of 10 time units. The goal is to evaluate how GBN handles high-error conditions in the network. The output reveals GBN's efficiency in sending multiple packets at once but also shows its vulnerability to cumulative issues when a single packet is lost or corrupted. After packet 17 was acknowledged, a corruption in packet 18 caused repeated retransmissions of packets 18–20, since GBN resends the entire window on timeout. The receiver continuously sent NAKs for the missing sequence number, but due to persistent corruption and loss, it took a long time to recover. This highlights that while GBN can improve throughput over Stop-and-Wait, its performance still degrades significantly under high corruption when earlier packets fail.

```
1 EVENT time: 739.442383, type: 2, fromlayer3 entity: 0
2 A_input: got NAK (ack = 20)...drop.
3 Simulator terminated at time 739.442383
4 after sending 20 msgs from layer5
```

Fig. 6. GBN Test 1 Results.

2. In Fig 7, this simulation tests the Go-Back-N (GBN) protocol with 20 messages, no packet loss or packet corruption and a message interval of 10 time units. The aim is to evaluate GBN's robustness in a corruption-only scenario. Despite no loss, persistent corruption of packet 20 and its ACKs caused the receiver to continuously send NAKs for sequence 19. As GBN requires in-order delivery, the sender repeatedly retransmitted packet 20, even though previous packets were delivered successfully. This led to excessive retransmissions and delayed completion. The protocol eventually succeeded, but the simulation illustrates how a single corrupted packet can stall the entire window in GBN, highlighting its inefficiency under persistent corruption without loss.



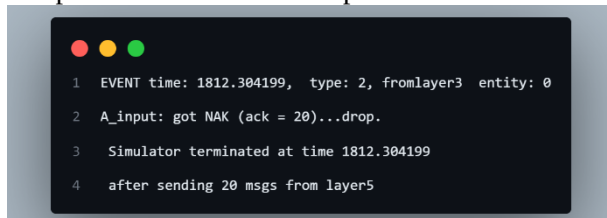
```

1 EVENT time: 727.169861, type: 2, fromlayer3 entity: 0
2 A_input: got NAK (ack = 20)...drop.
3 Simulator terminated at time 727.169861
4 after sending 20 msgs from layer5

```

Fig. 7. GBN Test 2 Results.

3. In Fig 7, this simulation tests the Go-Back-N (GBN) protocol with 20 messages, no packet loss, but with 20% packet corruption and a message interval of 10 time units. The aim is to evaluate GBN's robustness in a corruption-only scenario. This output simulates the Go-Back-N (GBN) protocol between two entities, A and B, with a focus on packet corruption and loss. Despite the protocol's error-handling mechanisms, a significant number of packets are corrupted due to a 0.2 corruption probability, leading to frequent negative acknowledgments (NAKs) from entity B. The high rate of packet corruption disrupts communication, with entity A simply dropping corrupted packets instead of retransmitting them, resulting in inefficiencies. Furthermore, repeated NAKs and dropped packets cause delays in the transmission process, reducing the overall throughput and making the communication less reliable. The simulation highlights how packet corruption and loss can significantly impact the performance of the GBN protocol.



```

1 EVENT time: 1812.304199, type: 2, fromlayer3 entity: 0
2 A_input: got NAK (ack = 20)...drop.
3 Simulator terminated at time 1812.304199
4 after sending 20 msgs from layer5

```

Fig. 8. GBN Test 3 Results.

IV. CONCLUSION

ABP and GBN provide foundational mechanisms for reliable data transfer, balancing simplicity and efficiency. ABP excels in low-bandwidth scenarios, while GBN's pipelining optimizes throughput. Validation under loss, corruption, and edge cases confirms their robustness, though practical deployments require enhancements like adaptive windowing and congestion control.